Porting React to Electron

Best of Both Worlds

`code/electron/electron_porting`

Why React + Electron?

- React's component system
- Modern development experience
- Desktop app capabilities
- File system access
- Native menus & notifications

Conversion Steps

1. **Keep React app as-is**
2. **Add Electron wrapper**
3. **Point Electron to React**
4. **Add desktop features**

Only three files are added or modified: `package.json`, `vite.config.js`, and `main.js`.

Electron Porting (Without Preload) Example

New Project Structure

```
electron_porting/
├── index.html
├── main.js
├── package.json
├── src
│   ├── App.css
│   ├── App.jsx
│   └── main.jsx
└── vite.config.js
```

1. Updated package.json

```
{
  "name": "electron-react-calculator",
  "main": "main.js",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "electron": "electron .",
    "start": "npm run build && electron .",
    "pack": "npm run build && electron-builder --dir",
    "dist": "npm run build && electron-builder"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.0.0",
    "electron": "^27.0.0",
    "electron-builder": "^24.0.0",
    "vite": "^4.3.0"
  },
  "build": {
    "appId": "com.example.calculator",
    "productName": "SimpleCalculator",
    "files": ["dist/**/*", "main.js", "package.json"],
    "directories": { "output": "release" }
  }
}
```

2. vite.config.js

The built `index.html` is loaded from a file (not from a server), so the asset paths must be **relative**: `base: './'`.

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  base: './',  // Important: Use relative paths for Electron
  build: {
    outDir: 'dist'
  }
})
```

3. main.js (Electron)

`npm start` first builds the React app (`vite build`). The `dist` directory then contains all the JavaScript files, including `index.html`.

```
const { app, BrowserWindow } = require('electron')
const path = require('path')

function createWindow() {
  const win = new BrowserWindow({
    width: 400,
    height: 600,
    webPreferences: {
      nodeIntegration: false,
      contextIsolation: true
    }
  })

  // Load the built React app from dist folder
  win.loadFile(path.join(__dirname, 'dist', 'index.html'))
}
```

The dist directory

```
dist
├── assets
│   ├── index-afa94370.css
│   └── index-eaaccfef.js
└── index.html
```

All the React library code and App business/UI logic is compiled in one JavaScript file (index-eaaccfef.js in this example)

```
<head>
  <meta charset="UTF-8">
  <script type="module" crossorigin src="./assets/index-eaaccfef.js"></script>
  <link rel="stylesheet" href="./assets/index-afa94370.css">
</head>
```

Electron Window Lifecycle Management

```
app.whenReady().then(() => {
  createWindow()
  app.on('activate', () => {
    if (BrowserWindow.getAllWindows().length === 0) {
      createWindow()
    }
  })
})

app.on('window-all-closed', () => {
  if (process.platform !== 'darwin') {
    app.quit()
  }
})
```

4. App.jsx

We can use the same `App.jsx` as in the React app.

```
import { useState } from 'react'

function App() {
  const [num1, setNum1] = useState('')
  const [num2, setNum2] = useState('')
  const [operation, setOperation] = useState('+')
  const [result, setResult] = useState(null)

  const calculate = () => {
    const n1 = parseFloat(num1)
    const n2 = parseFloat(num2)

    if (isNaN(n1) || isNaN(n2)) {
      setResult('Invalid input')
      return
    }
```

Development Workflow

```
# Install all dependencies
npm install

# Build React (dist/) and open it in Electron
npm start

# Optional: develop the React app in a browser (hot reload)
npm run dev
```

Production Build

```
# 1. Build React app
npm run build

# 2. Package with Electron (the script runs the build first)
npm run dist       # or: npm run pack (unpacked app, for testing)

# Output in release/ (names depend on the OS), for example:
#   SimpleCalculator-1.0.0.dmg          (macOS)
#   SimpleCalculator Setup 1.0.0.exe    (Windows)
#   SimpleCalculator-1.0.0.AppImage     (Linux)
```

The script `build.sh` runs these steps for you.